

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Multicores and Multiprogramming (Part 3)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Parallel computation complexity
- Flynn's Taxonomy
- Multithreading
- Parallel programming

This Lecture

- Loop dependences
- Shared memory and message passing
- Chip multiprocessors
- Parallel applications

Parallel Programming

Key design decisions:

Extracting parallelism/concurrency

- Pthreads, OpenMP

Communicating data

- Shared memory, message passing

Synchronizing computations

- Locks, barriers

Parallel Loops

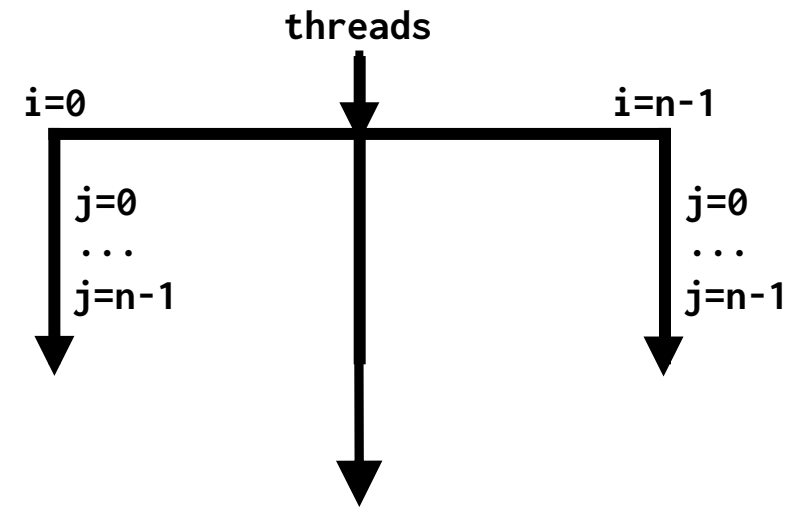
Parallelizing outer vs. inner loops:

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        b[i][j] = a[i][j];  
    }  
}
```

Parallel Loops

Parallelizing outer vs. inner loops:

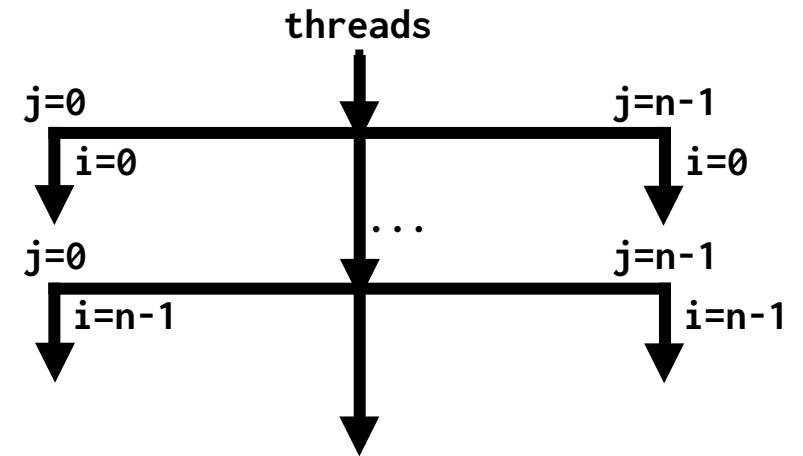
```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        b[i][j] = a[i][j];
    }
}
```



Parallel Loops

Parallelizing outer vs. inner loops:

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        b[i][j] = a[i][j];  
    }  
}
```



Loop Dependences

Are any of these loops parallelizable?

```
for(int i = 0; i < n; i++) {  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

Are any of these loops parallelizable?

```
#pragma omp parallel for?  
for(int i = 0; i < n; i++) {  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

Are any of these loops parallelizable?

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for?  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

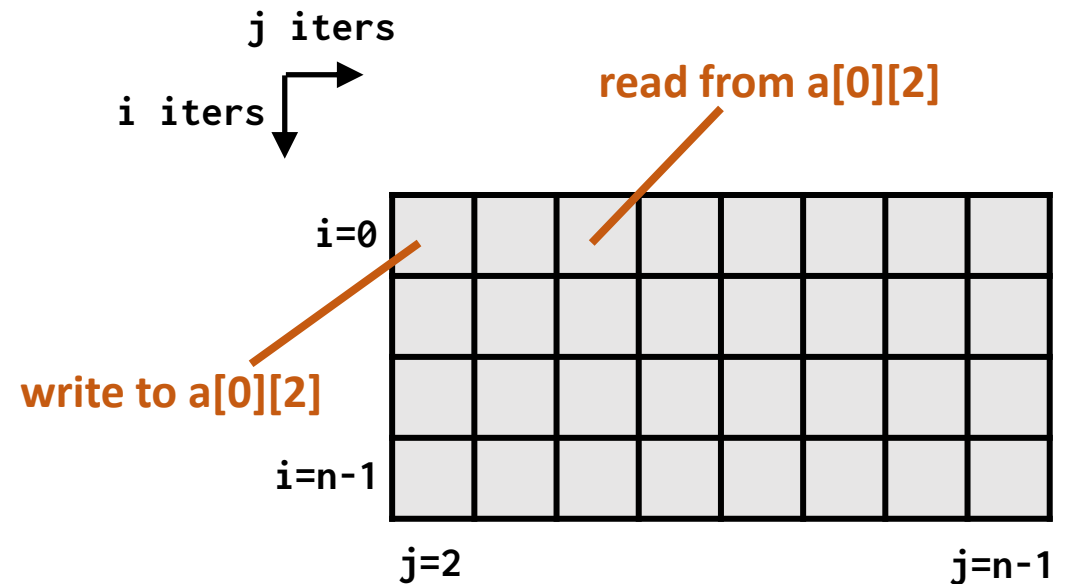
Are any of these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```

Loop Dependences

Are any of these loops parallelizable?

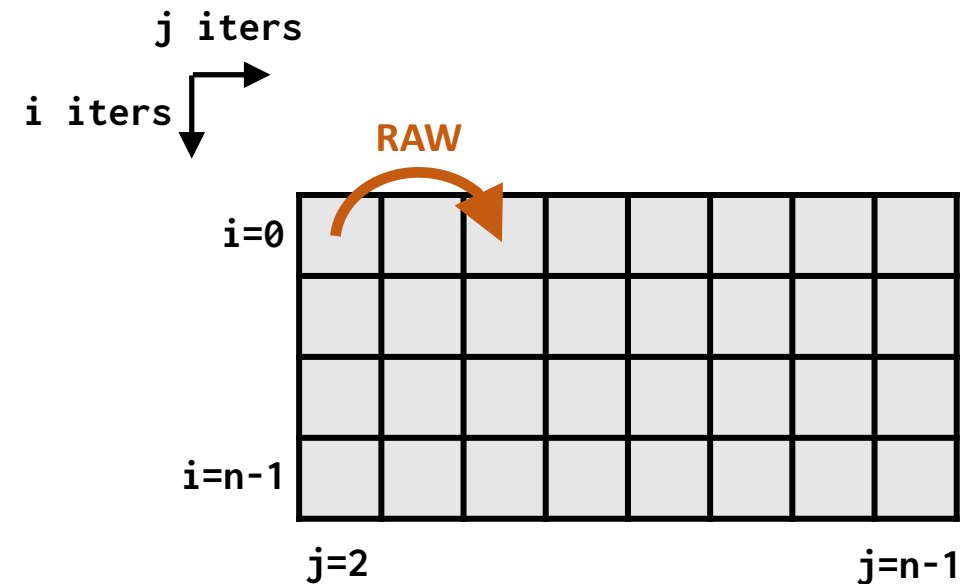
```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```



Loop Dependences

Are any of these loops parallelizable?

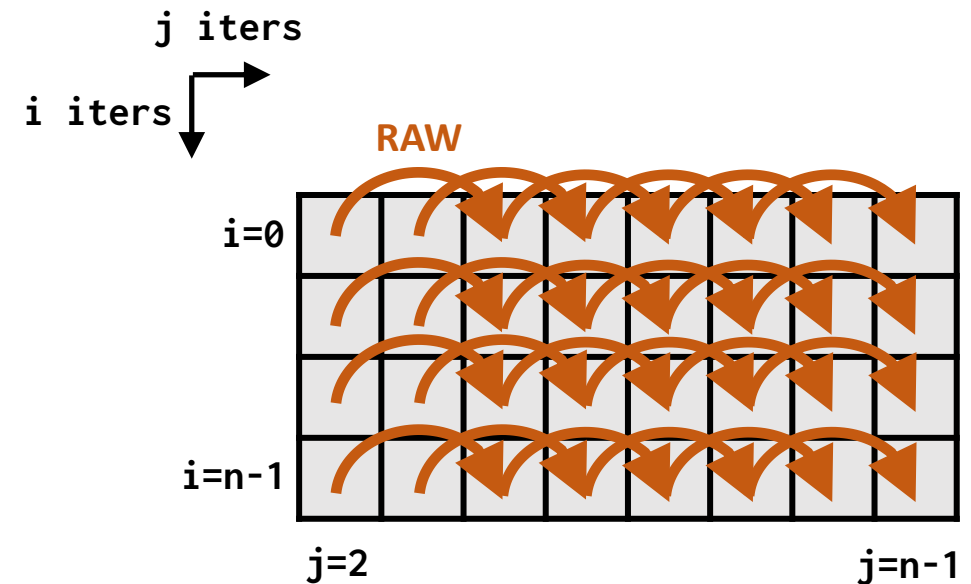
```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```



Loop Dependences

Are any of these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```



Loop Dependences

Is this loop parallelizable?

```
for(int i = 0; i < n-1; i++) {  
    a[i] = a[i+1];  
}
```

Loop Dependences

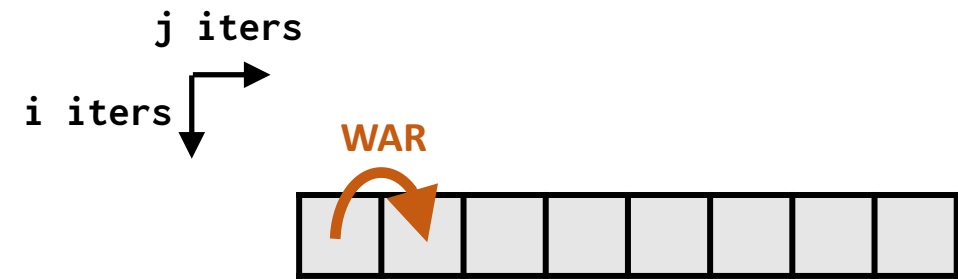
Is this loop parallelizable?

```
#pragma omp parallel for  
for(int i = 0; i < n-1; i++) {  
    a[i] = a[i+1];  
}
```

Loop Dependences

Is this loop parallelizable?

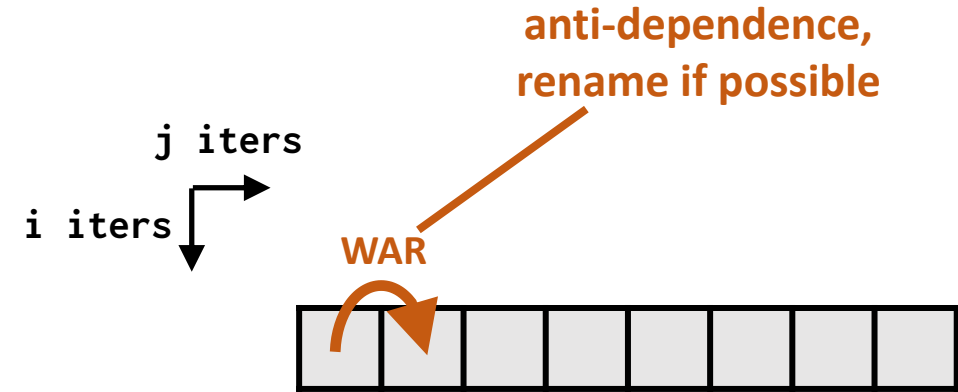
```
#pragma omp parallel for  
for(int i = 0; i < n-1; i++) {  
    a[i] = a[i+1];  
}
```



Loop Dependences

Is this loop parallelizable?

```
#pragma omp parallel for  
for(int i = 0; i < n-1; i++) {  
    a[i] = a[i+1];  
}
```



Loop Dependences

Is this loop parallelizable?

```
for(int i = 0; i < n-1; i++) {  
    b[i] = a[i+1];  
}
```

Loop Dependences

Is this loop parallelizable?

```
#pragma omp parallel for  
for(int i = 0; i < n-1; i++) {  
    b[i] = a[i+1];  
}
```

Loop Dependences

Is this loop parallelizable?

```
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
#pragma omp parallel for  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
#pragma omp parallel for  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

output dependence,
privatize if possible

Loop Dependences

Is this loop parallelizable?

```
int t;  
  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
  
for(int i = 0; i < n; i++) {  
    int t_local = a[i];  
    a[i] = b[i];  
    b[i] = t_local;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
#pragma omp parallel for  
for(int i = 0; i < n; i++) {  
    int t_local = a[i];  
    a[i] = b[i];  
    b[i] = t_local;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
#pragma omp parallel for  
for(int i = 0; i < n; i++) {  
    int t_local = a[i];  
    a[i] = b[i];  
    b[i] = t_local;  
}  
t = a[n-1];
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
int t;  
#pragma omp parallel for private(t)  
for(int i = 0; i < n; i++) {  
    t = a[i];  
    a[i] = b[i];  
    b[i] = t;  
}
```

Loop Dependences

Is this loop parallelizable?

```
for (int i = 0; i < N; i++) {  
    sum += a[i];  
}
```

Loop Dependences

Is this loop parallelizable?

```
#pragma omp parallel for reduction(+,sum)
for (int i = 0; i < N; i++) {
    sum += a[i];
}
```

Loop Dependences

Is this loop parallelizable?

```
d = 1;

for(int i = 0; i < n; i++) {
    b[i] = a[i + foo(d)];
    a[i] = c[i];
}
```

Loop Dependences

Is this loop parallelizable?

```
d = 1;
```

```
for(int i = 0; i < n; i++) {  
    b[i] = a[i + foo(d)];  
    a[i] = c[i];  
}
```

```
int foo(int x) {  
    ...  
    if (x > 0) return 0;  
}
```

Loop Dependences

Is this loop parallelizable?

```
d = 1;
```

```
for(int i = 0; i < n; i++) {  
    b[i] = a[i + foo(d)];  
    a[i] = c[i];  
}
```

```
int foo(int x) {  
    ...  
    if (x > 0) return 0;  
}
```

**compilers are conservative,
inline if possible**

Loop Dependences

Is this loop parallelizable?

```
d = 1;
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    int x;
    ...
    if (d > 0) x = 0;
    b[i] = a[i + x];
    a[i] = c[i];
}
```

Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

Is the i loop parallelizable? Is the j loop parallelizable?

Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

RAW for accesses to a[][]

RAW for accesses to b[][]

Is the i loop parallelizable? Is the j loop parallelizable?

Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

RAW for accesses to a[][]

RAW for accesses to b[][]

Is the i loop parallelizable? Is the j loop parallelizable?

No and no

Loop Dependences

How can the programmer/compiler increase parallelism?

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

Loop Dependences

How can the programmer/compiler increase parallelism?

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
    }  
}
```

loop fission

```
#pragma omp parallel for  
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        b[i][j] = b[i][j-1];  
    }  
}
```



Loop Dependences

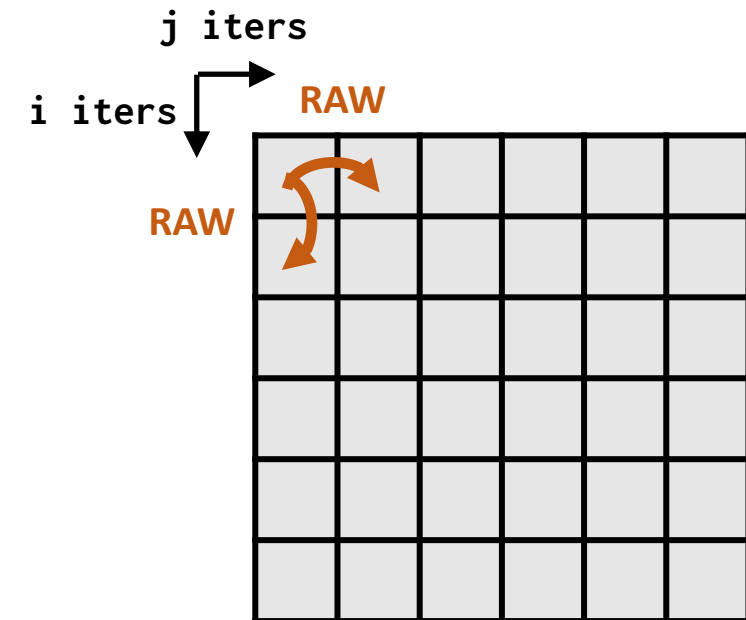
How can the programmer/compiler increase parallelism?

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

Loop Dependences

How can the programmer/compiler increase parallelism?

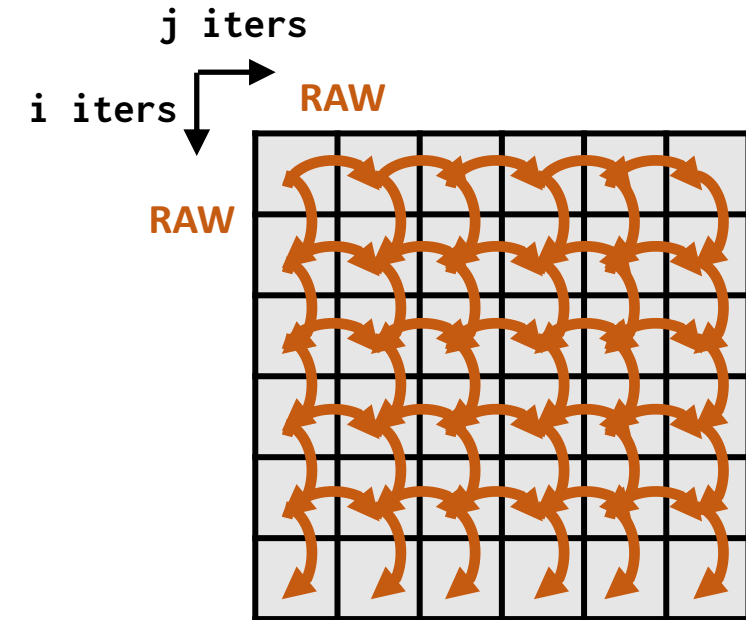
```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```



Loop Dependences

How can the programmer/compiler increase parallelism?

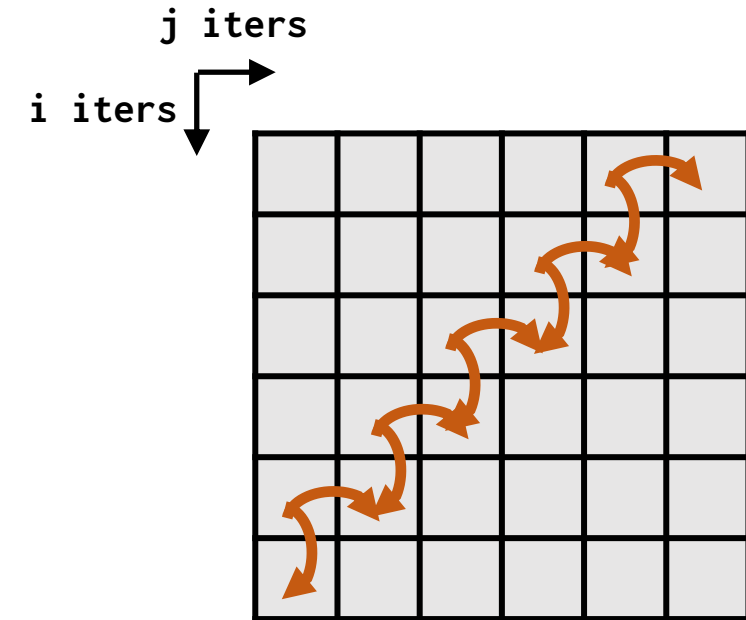
```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```



Loop Dependences

How can the programmer/compiler increase parallelism?

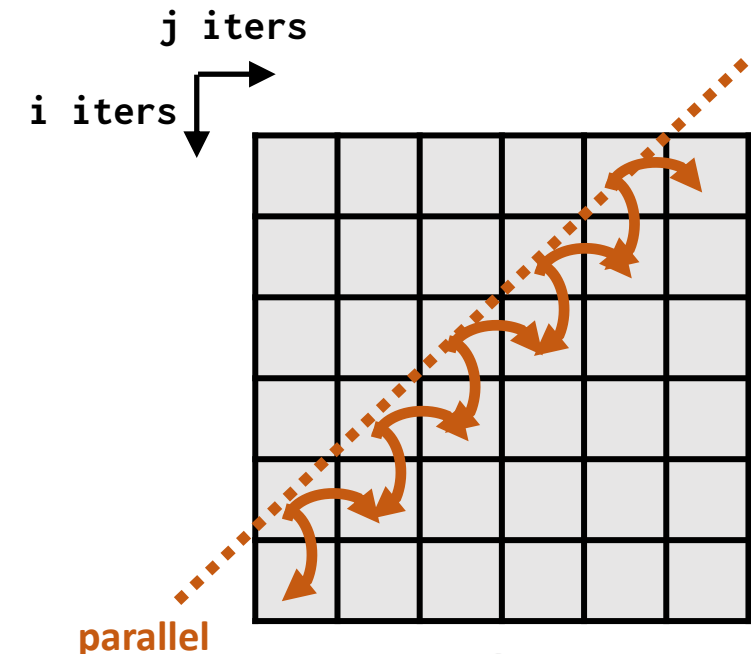
```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```



Loop Dependences

How can the programmer/compiler increase parallelism?

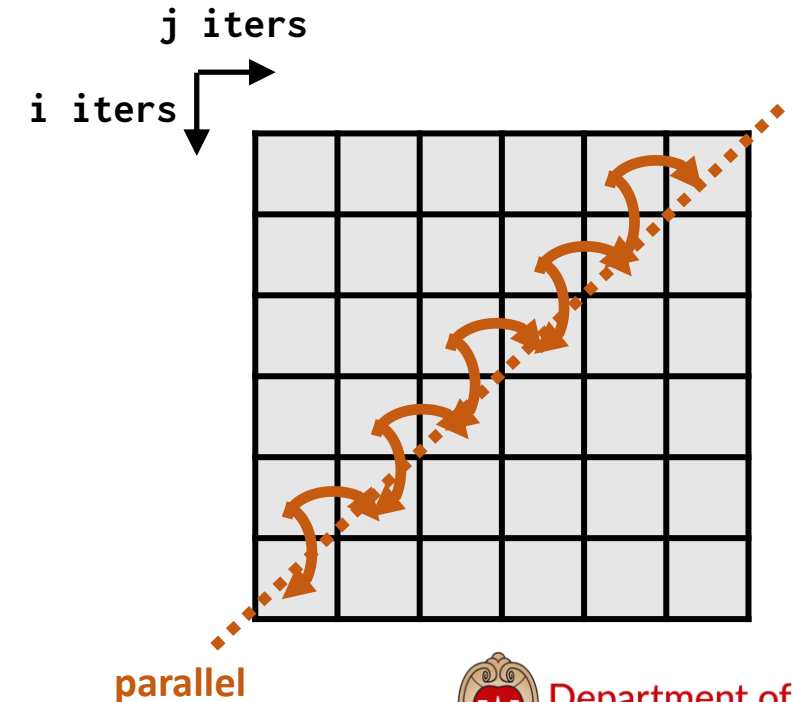
```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```



Loop Dependences

How can the programmer/compiler increase parallelism?

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i-1][j];  
        b[i][j] = b[i][j-1];  
    }  
}
```

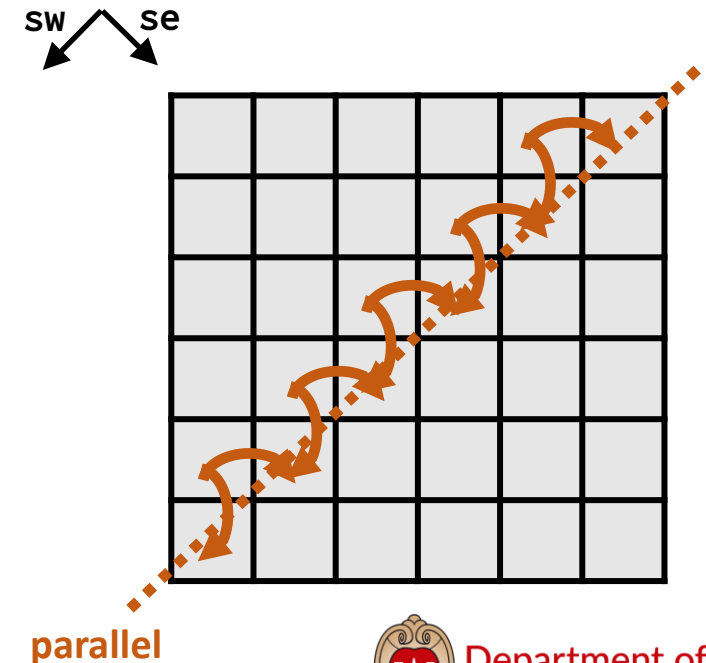


Loop Dependences

How can the programmer/compiler increase parallelism?

```
for(int se = 0; se < n+n-1; se++) {  
    #pragma omp parallel for  
    for(int sw = max(0, se-n-1); sw < min(n, se+1); sw++) {  
        a[sw][se-sw] = a[sw-1][se-sw];  
        b[sw][se-sw] = b[sw][se-sw-1];  
    }  
}
```

loop skewing



Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

Is the i loop parallelizable? Is the j loop parallelizable?

Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

RAW for accesses to c[][]

Is the i loop parallelizable? Is the j loop parallelizable?

Loop Dependences

List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

RAW for accesses to c[][]

Is the i loop parallelizable? Is the j loop parallelizable?

No and yes

Loop Dependences

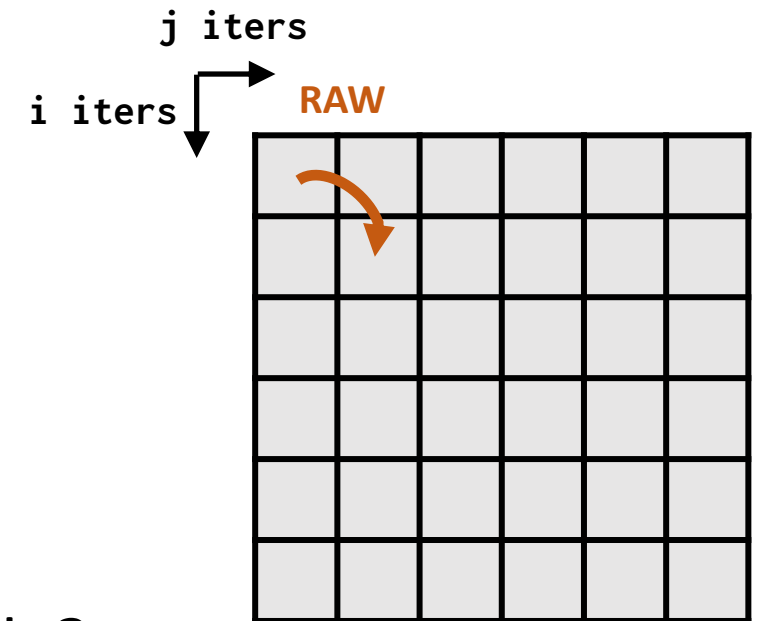
List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

RAW for accesses to c[][]

Is the i loop parallelizable? Is the j loop parallelizable?

No and yes



Loop Dependences

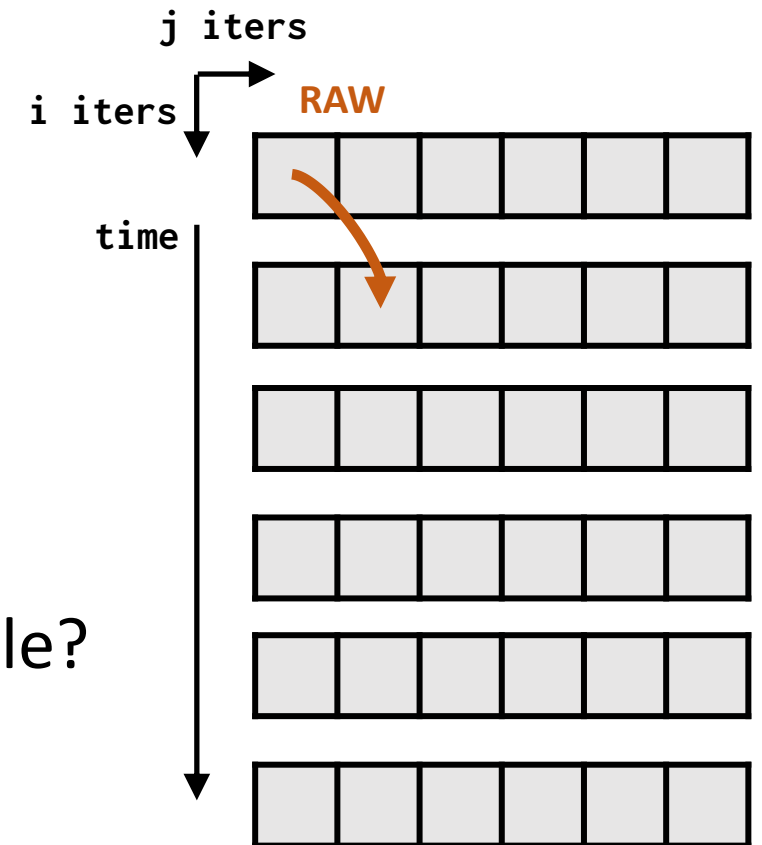
List the loop-carried dependences (if any) in the following loop nest.

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

RAW for accesses to c[][]

Is the i loop parallelizable? Is the j loop parallelizable?

No and yes



Loop Dependences

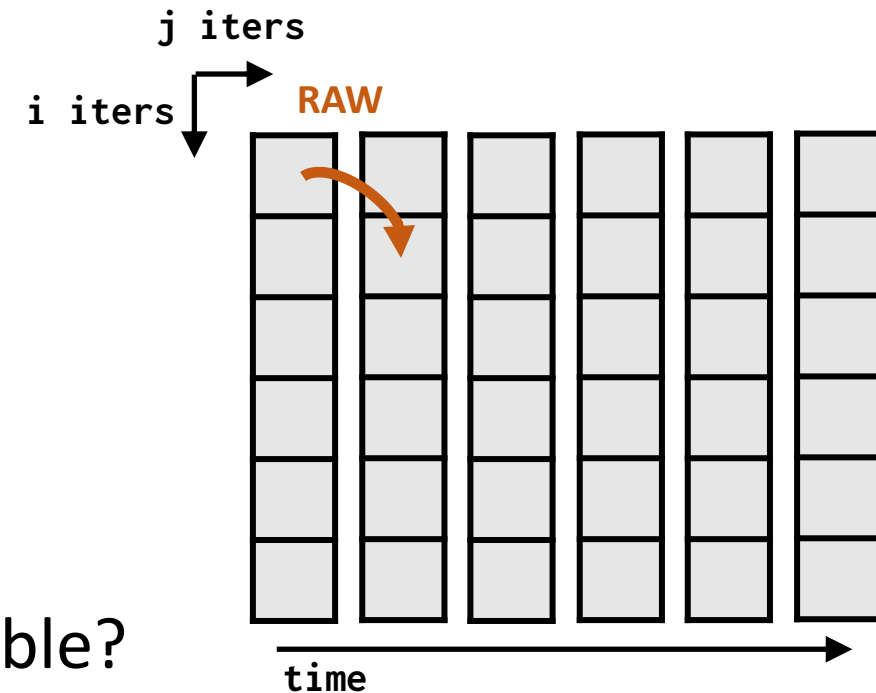
List the loop-carried dependences (if any) in the following loop nest.

```
for(int j = 0; j < n; j++) {  
    #pragma omp parallel for  
    for(int i = 0; i < n; i++) {  
        c[i][j] = c[i-1][j-1];  
    }  
}
```

RAW for accesses to c[][]

Is the i loop parallelizable? Is the j loop parallelizable?

Innermost loop is parallelizable



Parallel Programming

Key design decisions:

Extracting parallelism/concurrency

- Pthreads, OpenMP

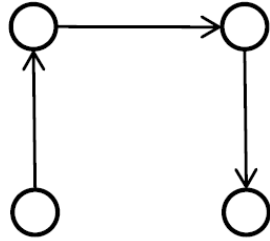
Communicating data

- Shared memory, message passing

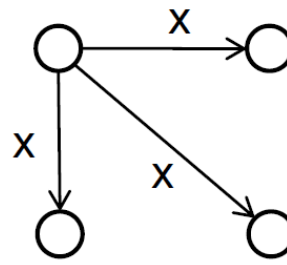
Synchronizing computations

- Locks, barriers

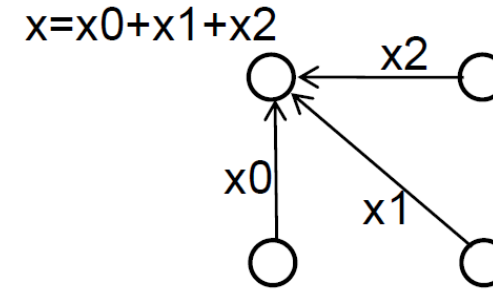
Common Communication Patterns



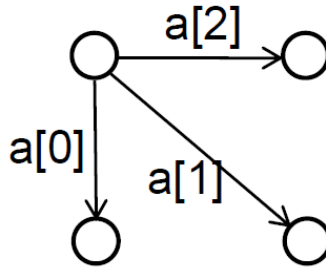
Point-to-point



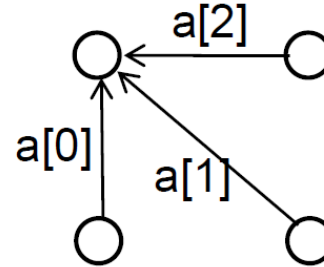
Broadcast



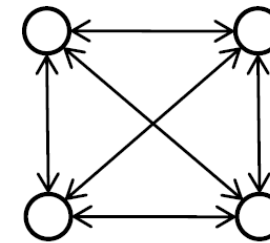
Reduce (eg., '+')



Scatter



Gather



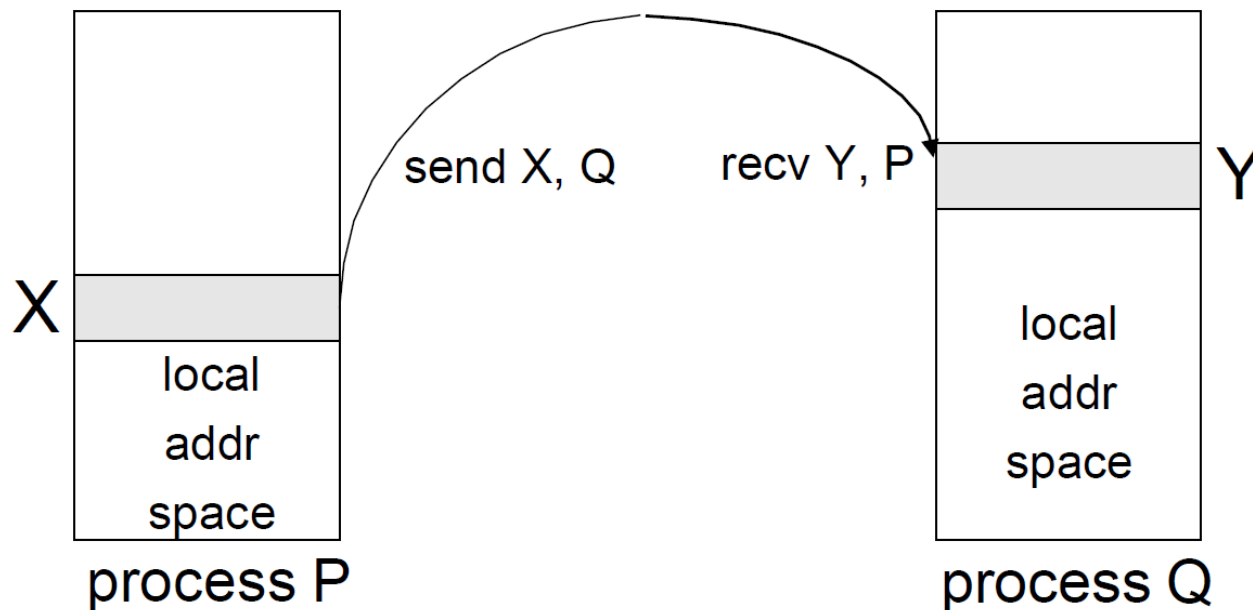
All-to-all



Message Passing Interface

API/protocol for explicit communication between processes:

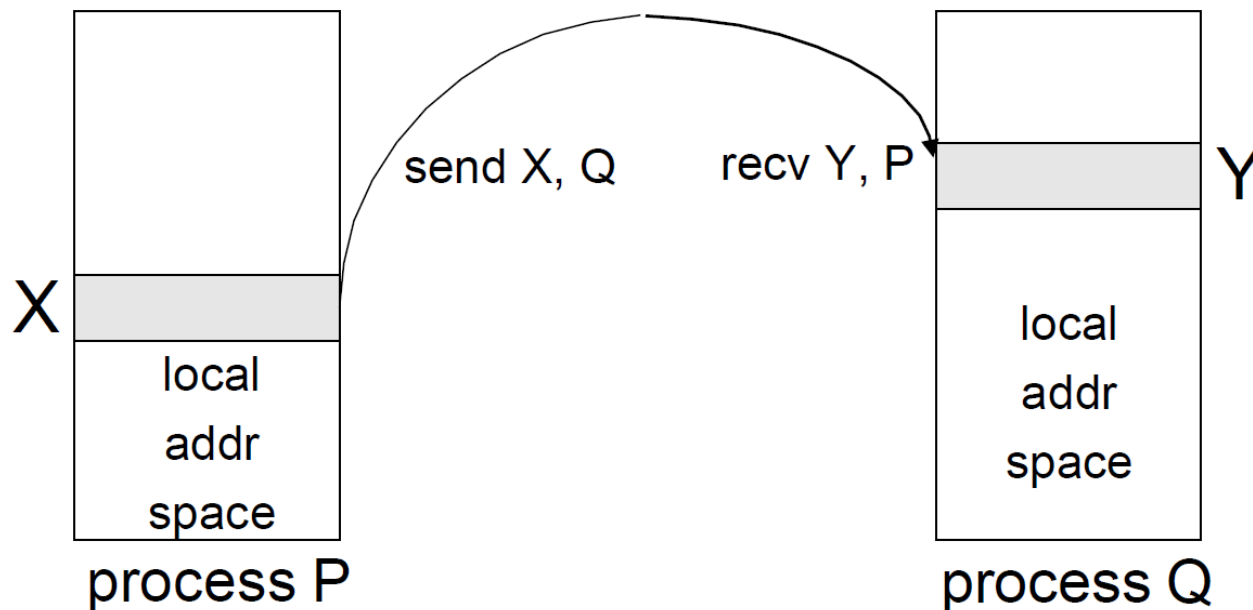
- Implemented via I/O (e.g., multi-chip clusters) or non-coherent memory buffers (e.g., Intel SCC)
- Supports synchronous/asynchronous, point-to-point/collective communication
- Provides data communication and synchronization in the same mechanism



Message Passing Interface

API/protocol for explicit communication between processes:

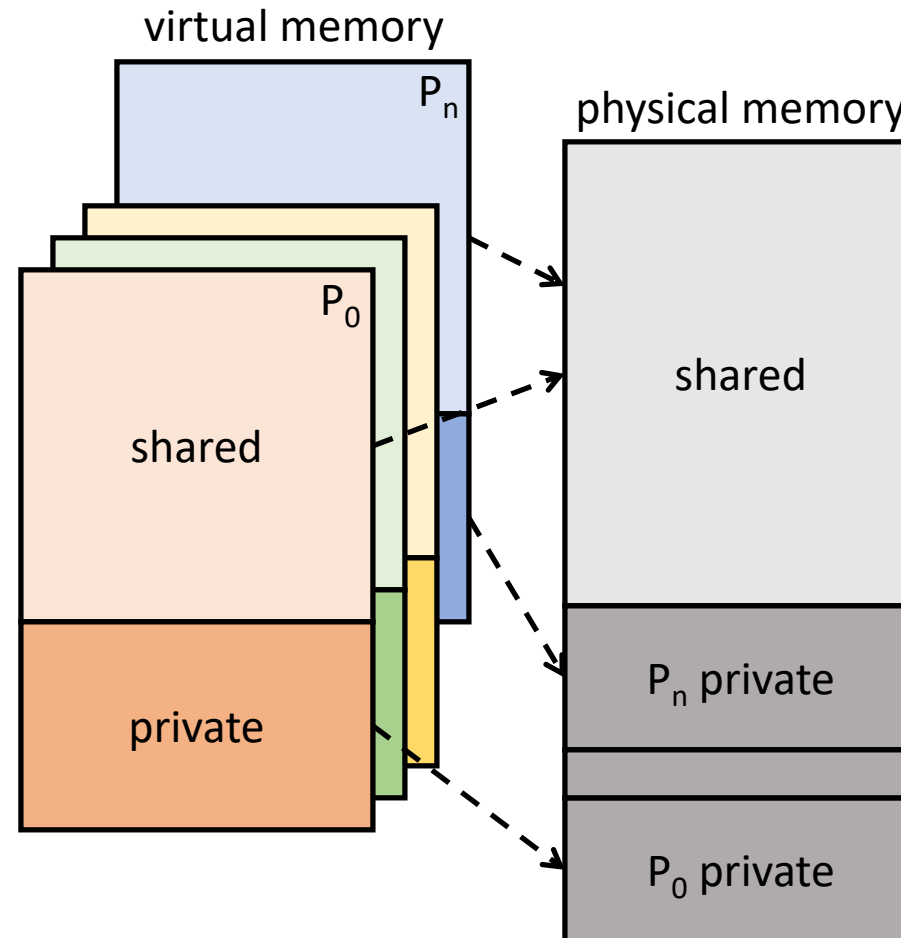
- Implemented via I/O (e.g., multi-chip clusters) or non-coherent memory buffers (e.g., Intel SCC)
- Supports synchronous/asynchronous, point-to-point/collective communication
- Provides data communication and synchronization in the same mechanism



MPI_Init, MPI_Finalize
MPI_Send, MPI_Recv
MPI_Bcast, MPI_Reduce
MPI_Gather, MPI_Scatter
MPI_Barrier

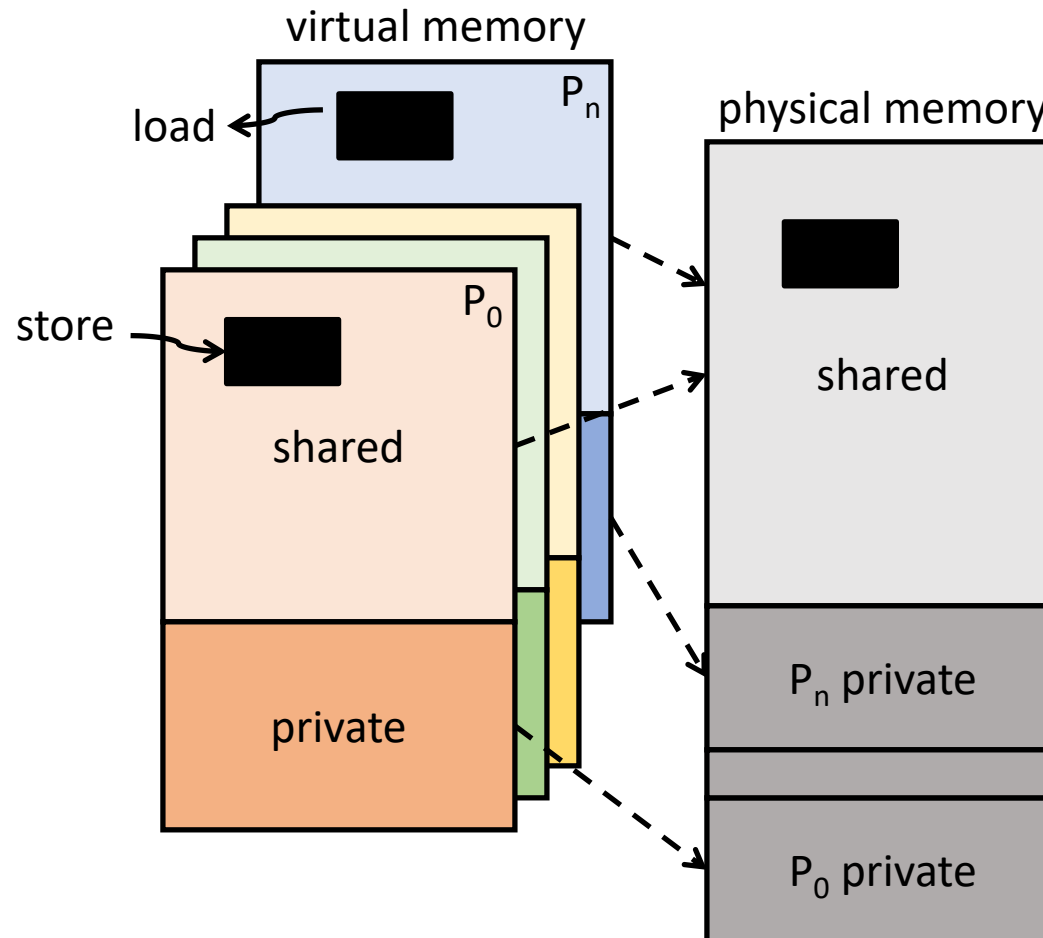
Shared Memory

Programming model for implicit communication between processes/threads.



Shared Memory

Programming model for implicit communication between processes/threads.



Shared Memory and Message Passing

Shared Memory

- + Low overhead per datum; good for fine-grained data movement
- + Supports uncommon communication patterns
- + Sending/receiving simple for programmers

vs

Message Passing

- + Can move large chunks of data; good for coarse-grained data movement
- + Supports off-chip, distributed communication
- + Synchronization simple for programmers

Shared Memory and Message Passing

Shared Memory

+ Low overhead per datum; good for fine-grained data movement

+ Supports uncommon communication patterns

Message Passing

+ Can move large chunks of data; good for coarse-grained data movement

+ Supports on-chip, distributed communication

An analogy:

Vacationing with relatives can be hectic.

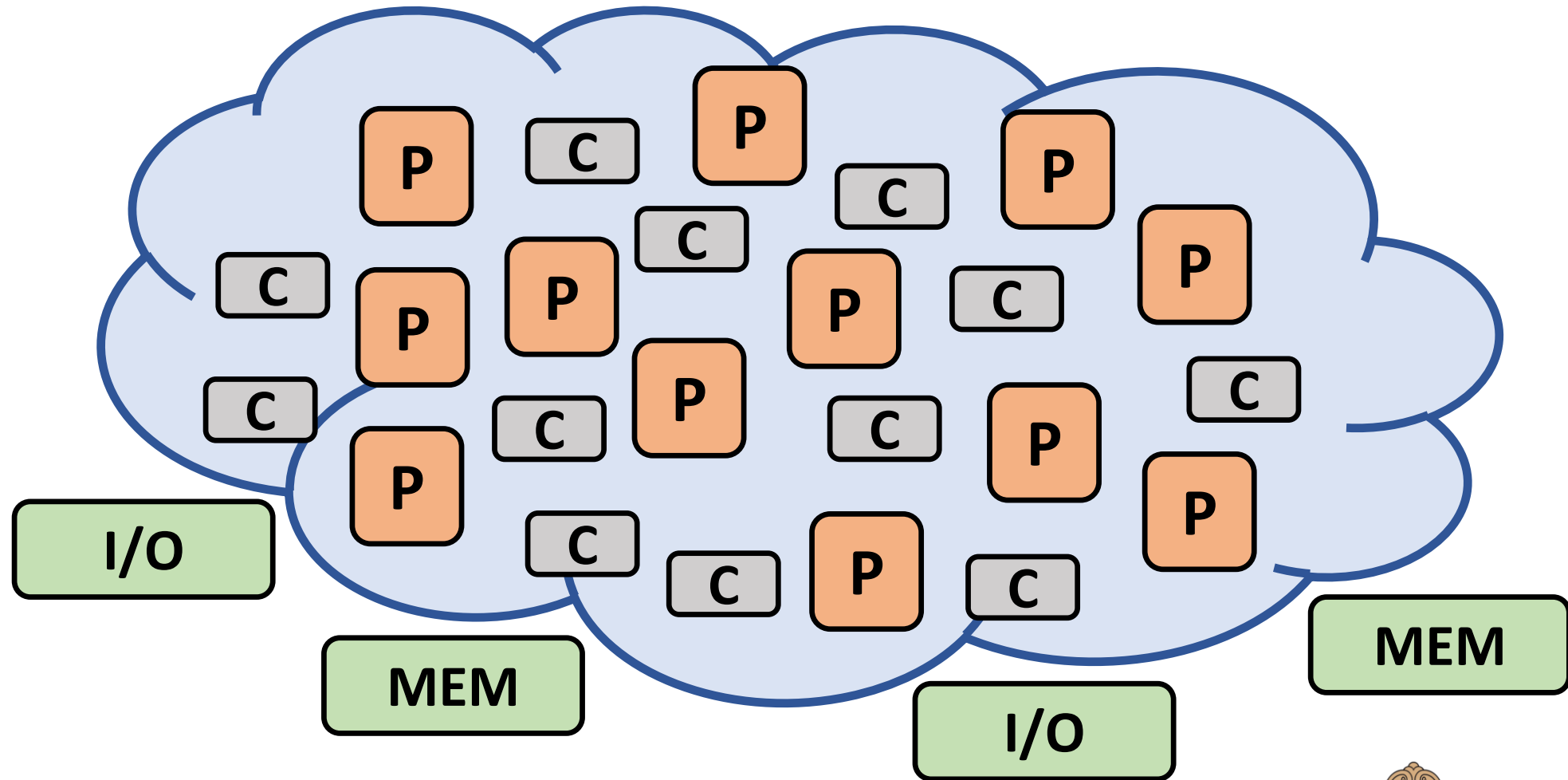
Each family driving their car is low-overhead and flexible, whereas taking the bus saves off-region tolls and makes sure everyone aligns to the scheduled itinerary.

programmers

programmers



Chip Multiprocessors

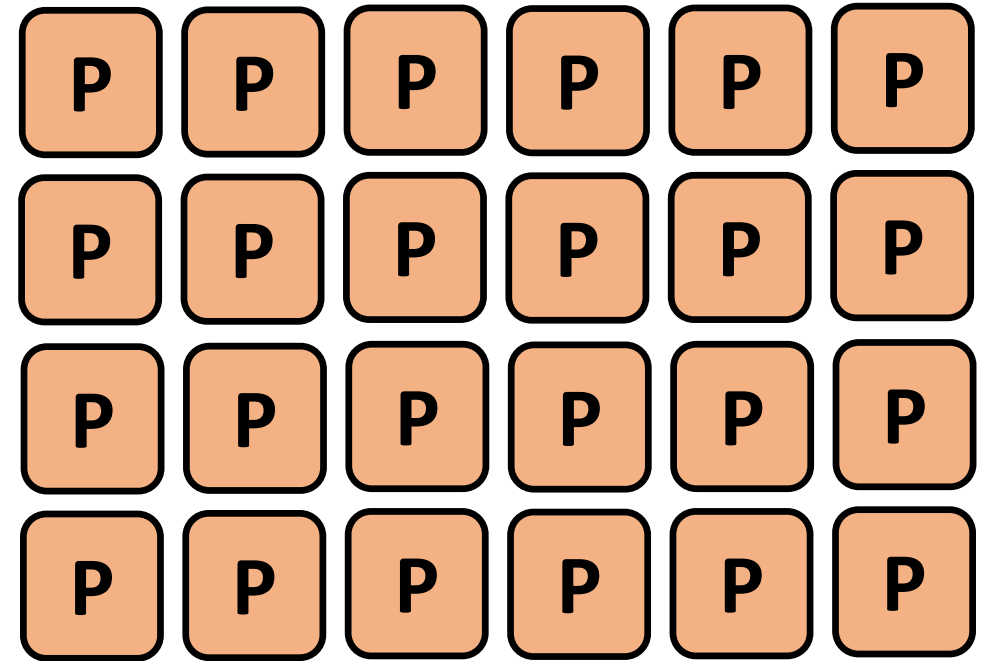


Homogeneous CMPs

All processor cores have same configuration (symmetric multiprocessors).

When to use:

- Well-suited for multithreaded applications with uniform work distribution (e.g., image processing, scientific, linear algebra)
- Simplifies programming: predictable performance
- Lower development cost: easy to stamp out identical cores
- Redundancy for fault tolerance (e.g., cloud)
- Can still mimic heterogeneity via DVFS and power-gating resources (e.g., cache banks)

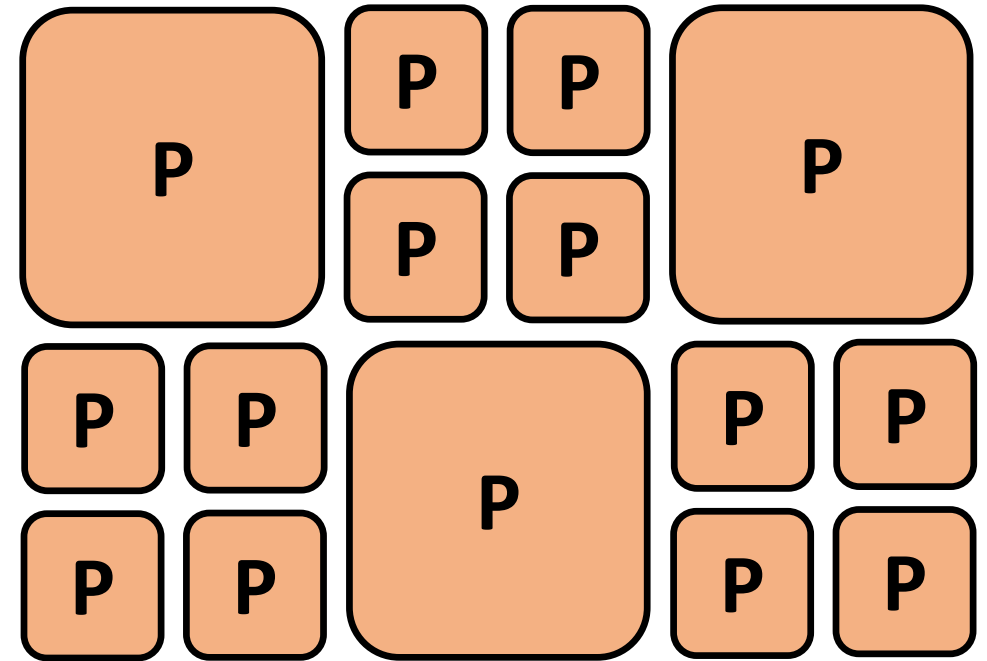


Heterogeneous CMPs

Processor cores have different configurations.

Sources of heterogeneity:

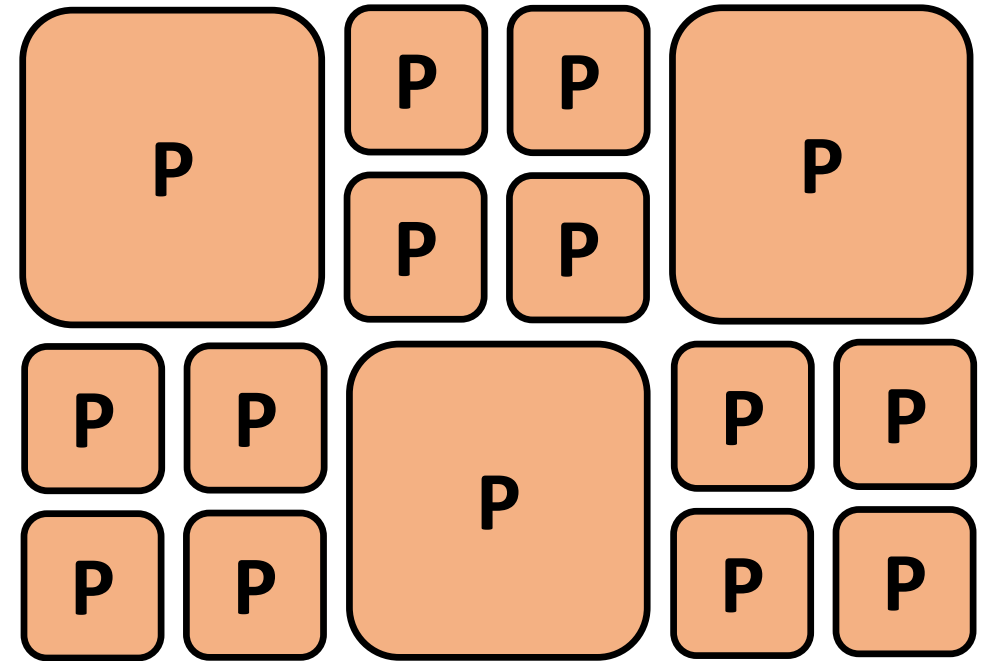
- Processor ISA (e.g., some RISC, some CISC)
- Core microarchitecture (e.g., some in-order, some out-of-order)
- Cache configurations (e.g., some high capacity, some low)
- Architectural optimizations (e.g., some with branch prediction, some without)



Heterogeneous CMPs

When to use:

- Well-suited for diverse mix of applications (e.g., cloud)
- Well-suited for multithreaded applications with load imbalance (e.g., graph analytics, physical simulation, producer-consumer)
- Well-suited for applications with distinct phases (e.g., sequential vs. parallel phases)
- Can specialize for dark silicon
- Offers more flexibility for expert software developers



Homogeneous vs. Heterogeneous Trade-Off

Architecture **generality** vs. **efficiency**:

Build homogeneous CMP for the average case
(i.e., maximize efficiency for the average application)?

OR

Build heterogeneous CMP for specific subset
(i.e., maximize efficiency for some specialized applications)?

Homogeneous vs. Heterogeneous Trade-Off

Programming **productivity** vs. **performance**:

Build homogeneous CMP with uniform, predictable programming platform?

OR

Build heterogeneous CMP with diverse, flexible programming platform?

CMP Challenges

Where to allocate threads on cores?

CMP Challenges

Where to allocate threads on cores?

- **Locality-based:** cluster closely communicating threads together
 - Reduces delay across interconnect and allows for sharing in caches

CMP Challenges

Where to allocate threads on cores?

- **Locality-based:** cluster closely communicating threads together
 - Reduces delay across interconnect and allows for sharing in caches
- **Memory-based:** place threads close to where their data is located
 - Off-chip memory is often banked: multiple memory controllers at different locations on the chip, each one with access to distinct subset of memory space
 - It's not just about being “close” to memory; need to spread out threads enough to not choke up interconnect bandwidth [B. Lepers et al., USENIX ATC 2015]

CMP Challenges

Where to allocate threads on cores?

- **Locality-based:** cluster closely communicating threads together
 - Reduces delay across interconnect and allows for sharing in caches
- **Memory-based:** place threads close to where their data is located
 - Off-chip memory is often banked: multiple memory controllers at different locations on the chip, each one with access to distinct subset of memory space
 - It's not just about being “close” to memory; need to spread out threads enough to not choke up interconnect bandwidth [B. Lepers et al., USENIX ATC 2015]
- **Power-based:** consolidate threads for lower power
 - Consolidating threads onto less cores (via either context-switching or SMT) saves power at the cost of performance; less active cores but less throughput
 - Apply heuristics to estimate power-performance trade-off and allocate threads accordingly [A. Vega et al., PACT 2013]

CMP Challenges

Where to allocate threads on cores?

- **Locality-based:** cluster closely communicating threads together
 - Reduces delay across interconnects for sharing in caches
- **Memory-based:** Consolidate passengers with Uber Pool.
Lower cost but everyone gets to their destination later.
 - Off-chip memory is often banked: multiple memory controllers at different locations on the chip, each one with access to distinct subset of memory space
 - It's not just about being "close" to memory; need to spread out threads enough to not choke up interconnect bandwidth [B. Lepers et al., USENIX ATC 2015]
- **Power-based:** consolidate threads for lower power
 - Consolidating threads onto less cores (via either context-switching or SMT) saves power at the cost of performance; less active cores but less throughput
 - Apply heuristics to estimate power-performance trade-off and allocate threads accordingly [A. Vega et al., PACT 2013]

CMP Challenges

When to migrate threads to other cores?

CMP Challenges

When to migrate threads to other cores?

- **Phase-based:** move threads to exploit specialized cores
 - Threads with distinct phase behavior can be moved to better suited cores

CMP Challenges

When to migrate threads to other cores?

- **Phase-based:** move threads to exploit specialized cores
 - Threads with distinct phase behavior can be moved to better suited cores
- **Age-based:** move slower/longer threads to faster cores
 - Multithreaded applications are only as fast as their slowest thread
 - Interactive services in datacenters impose requirements on request tail latencies (99th-percentile); age-based migration to faster cores can better satisfy tail latency requirements than age-based DVFS [M. E. Haque et al., MICRO 2017]

CMP Challenges

When to migrate threads to other cores?

- **Phase-based:** move threads to exploit specialized cores
 - Threads with distinct phase behavior can be moved to better suited cores
- **Age-based:** move slower/longer threads to faster cores
 - Multithreaded applications are only as fast as their slowest thread
 - Interactive services in datacenters impose requirements on request tail latencies (99th-percentile); age-based migration to faster cores can better satisfy tail latency requirements than age-based DVFS [M. E. Haque et al., MICRO 2017]
- **Reuse-based:** move threads to where they can borrow warm resources
 - Threads can warm up caches or branch predictors for other similar threads
 - Transaction queries have huge instruction footprints but also have common computation phases; once a query starts a new phase, move it to a core with a pre-filled instruction cache [I. Atta et al., MICRO 2013]

CMP Challenges

When to migrate threads to other cores?

- **Phase-based:** move threads to exploit specialized cores
 - Threads with distinct phase behavior can be moved to better suited cores
- **Age-based:** move slower/longer threads to faster cores
 - Multithreaded applications are only as fast as their slowest thread
 - Interactive services in datacenters impose requirements on request tail latencies (99th-percentile); age-based migration to faster cores can better satisfy tail latency requirements than age-based DVFS [M. E. Haque et al., MICRO 2017]
- **Reuse-based:** move threads to where they can borrow warm resources
 - Threads can warm up caches or branch predictors for other similar threads
 - Transaction queries have huge instruction footprints but also have common computation phases; once a query starts a new phase, move it to a core with a pre-filled instruction cache [I. Atta et al., MICRO 2013]

CMP Challenges

How and where to allocate data storage?

- Cache and memory hierarchy
 - Coming soon...

How to facilitate data sharing?

- Synchronization, memory consistency
 - Coming soon...

How and where to move data?

- Cache coherence, network-on-chip
 - Coming soon...

How to integrate specialized cores?

- GPUs, Accelerators
 - Coming soon...

Evaluation Workloads

Computation kernels:

- Exercise fundamental set of algorithms and communication patterns

End-to-end applications:

- Represent important/common workloads from the real world

Benchmarks:

- Balance the above two

Parallel Benchmark Suites

Multiprogrammed:

SPEC CPU 2017

- industry standard, compute-intensive, integer and floating-point

Multithreaded:

PARSEC 3.0

- diverse synchronization and sharing patterns

SPLASH-2

- compute-intensive workloads for high-performance computing

PERFECT 1.0

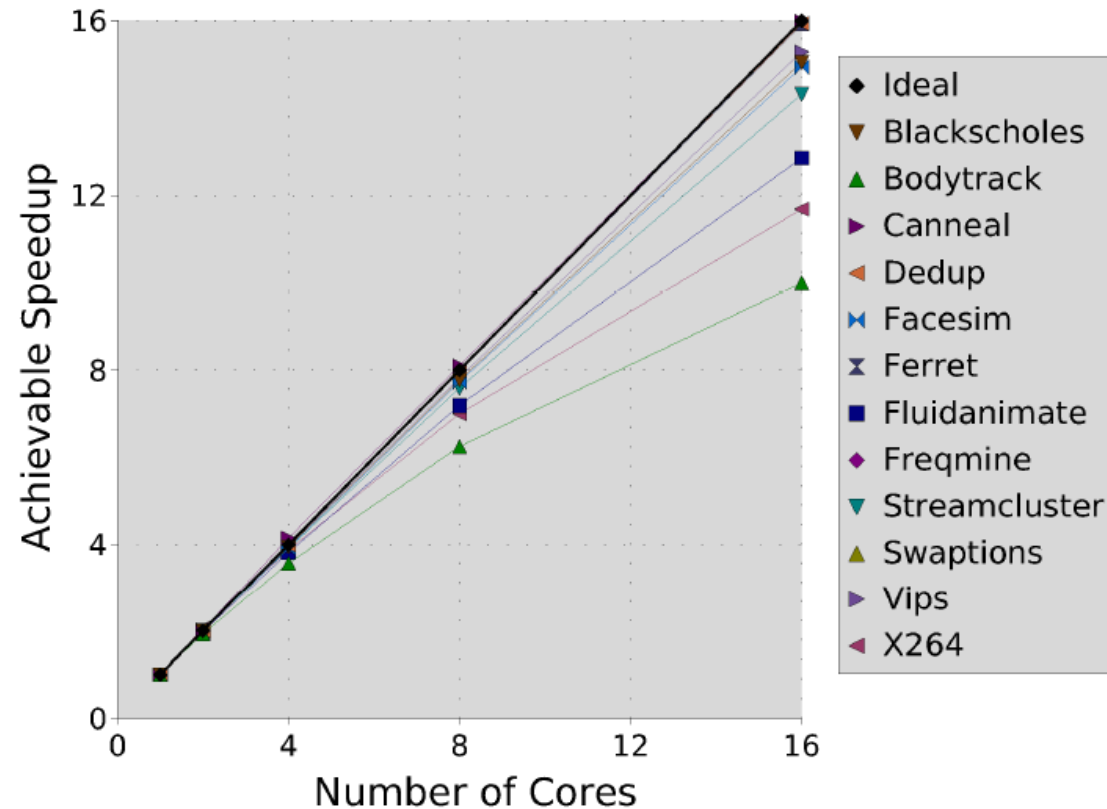
- computer vision, image processing

CloudSuite 3.0

- online services, data-centric, massive working sets

Benchmark Considerations

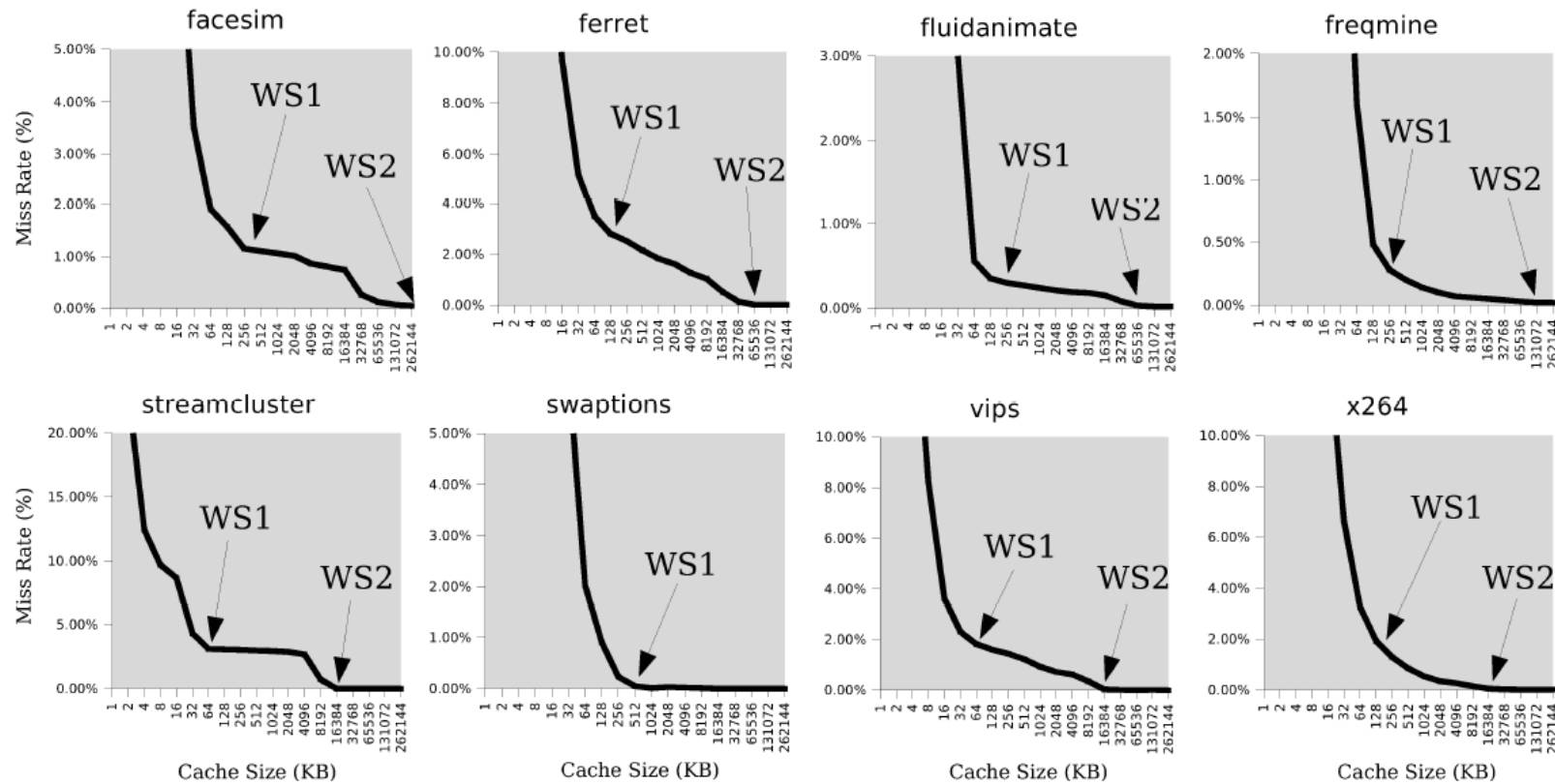
Upper-bound parallel speedup?



C. Bienia et al., *The PARSEC Benchmark Suite: Characterization and Architectural Implications*, PACT, 2008.

Benchmark Considerations

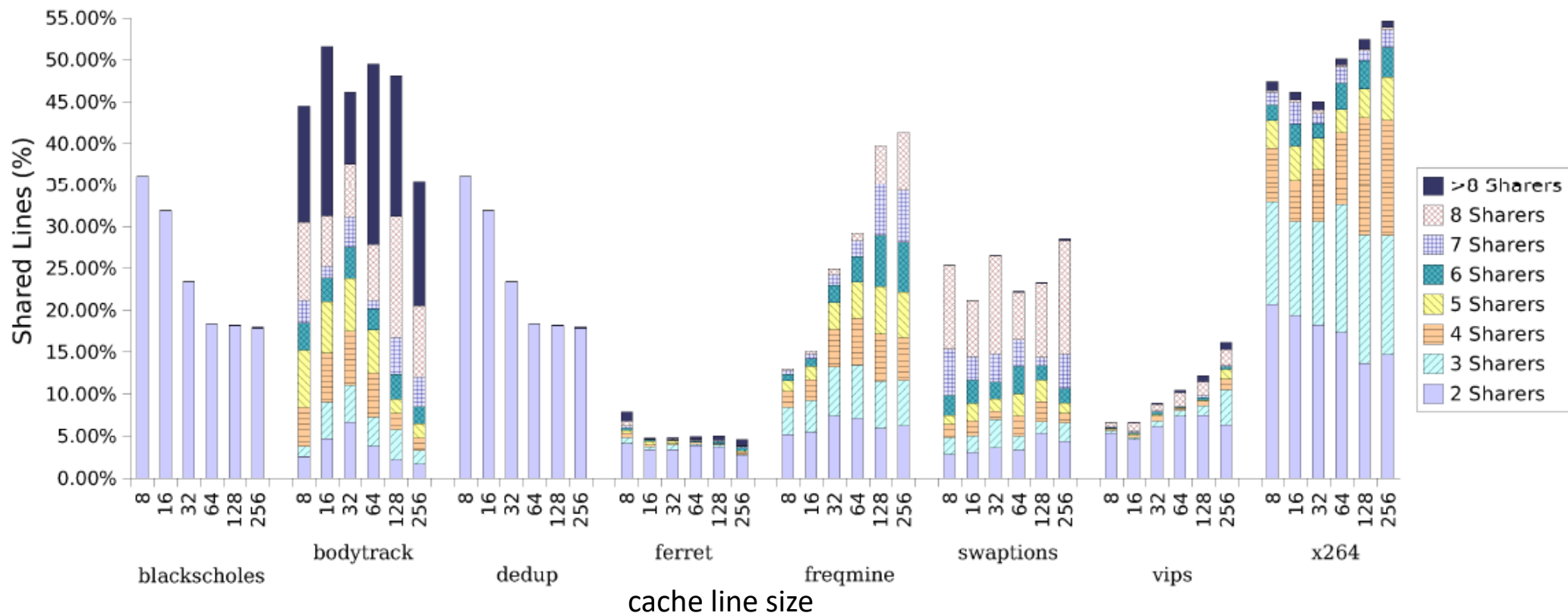
Working set size vs. cache size?



C. Bienia et al., *The PARSEC Benchmark Suite: Characterization and Architectural Implications*, PACT, 2008.

Benchmark Considerations

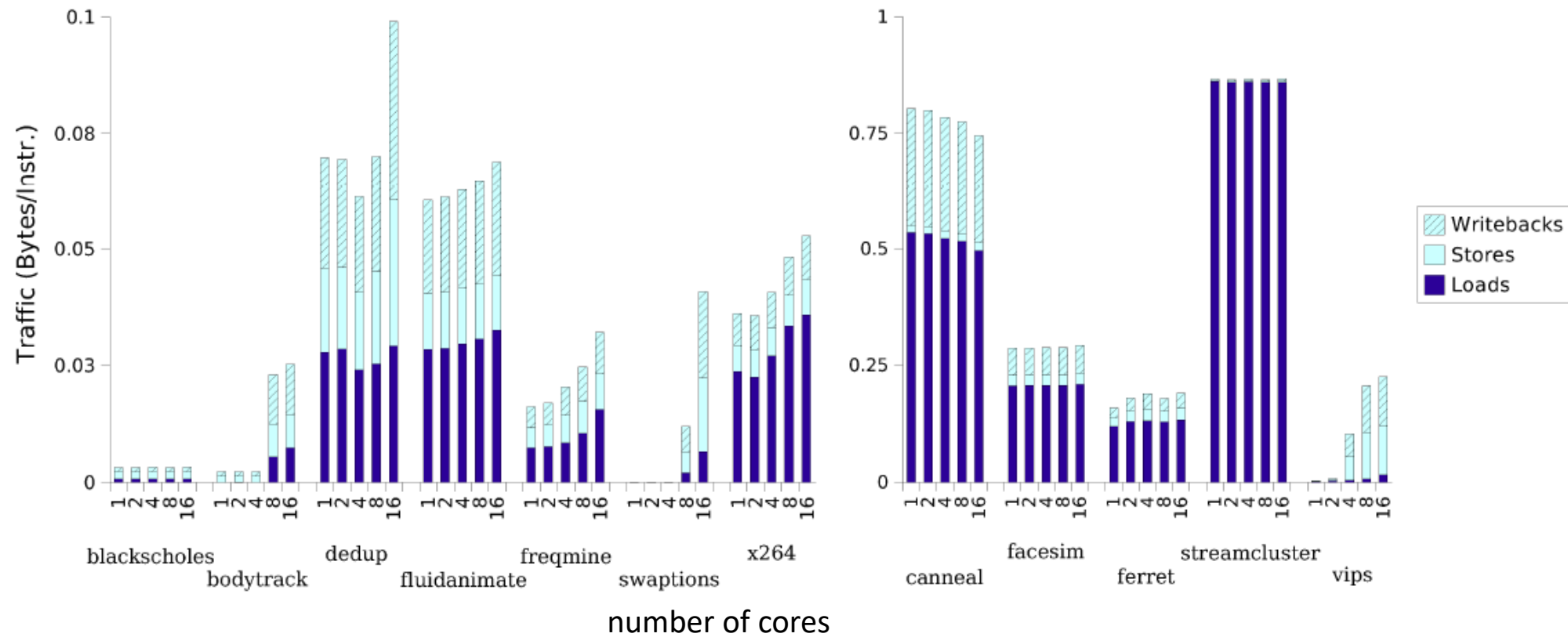
Amount of data sharing amongst threads?



C. Bienia et al., *The PARSEC Benchmark Suite: Characterization and Architectural Implications*, PACT, 2008.

Benchmark Considerations

Interconnect traffic?

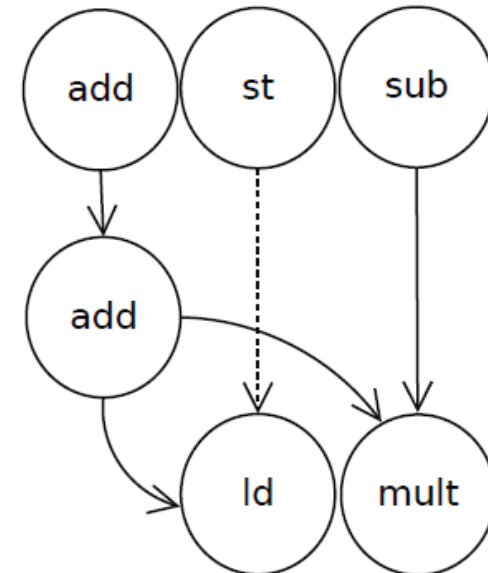
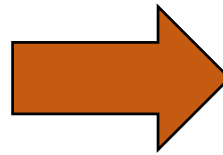
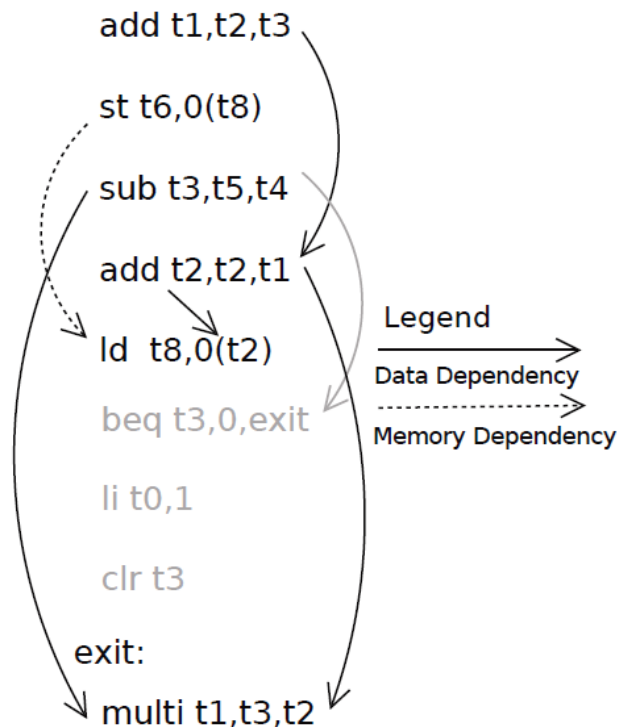


C. Bienia et al., *The PARSEC Benchmark Suite: Characterization and Architectural Implications*, PACT, 2008.

Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

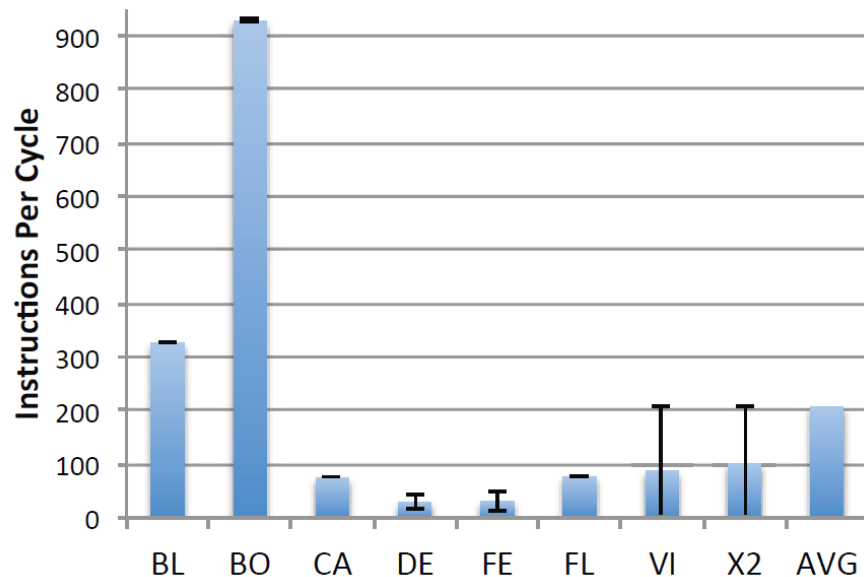
Build data flow graphs from execution traces to estimate fundamental amount of compute-level parallelism in multithreaded applications with and without synchronization.



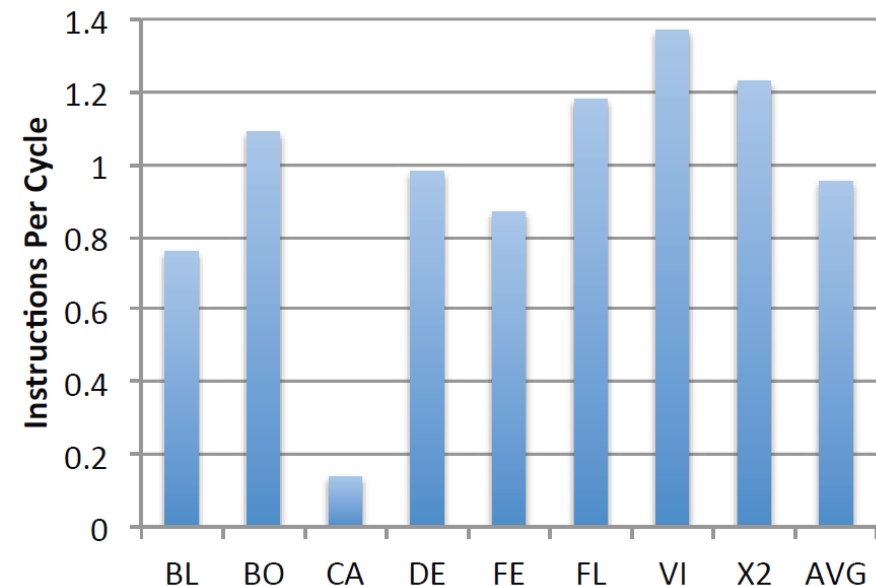
Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

upper bound ILP (2 threads)



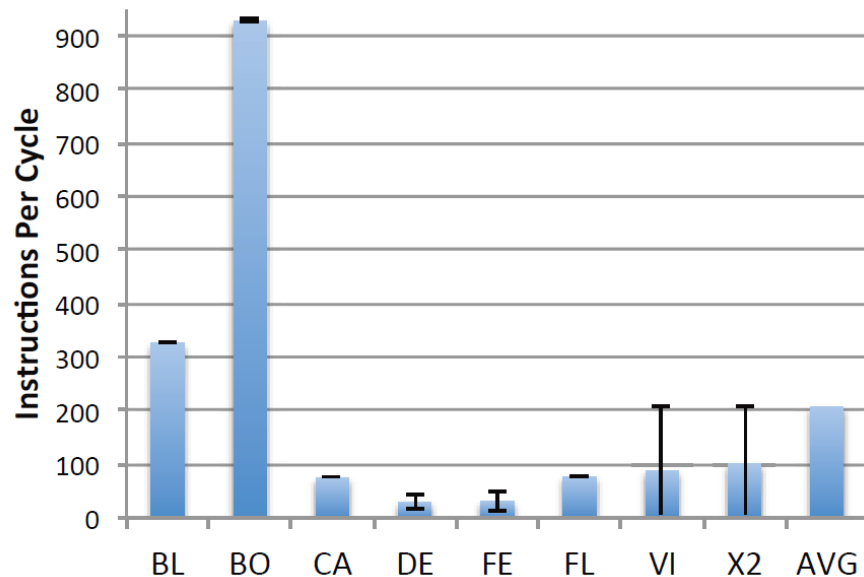
real hardware ILP (2 threads)



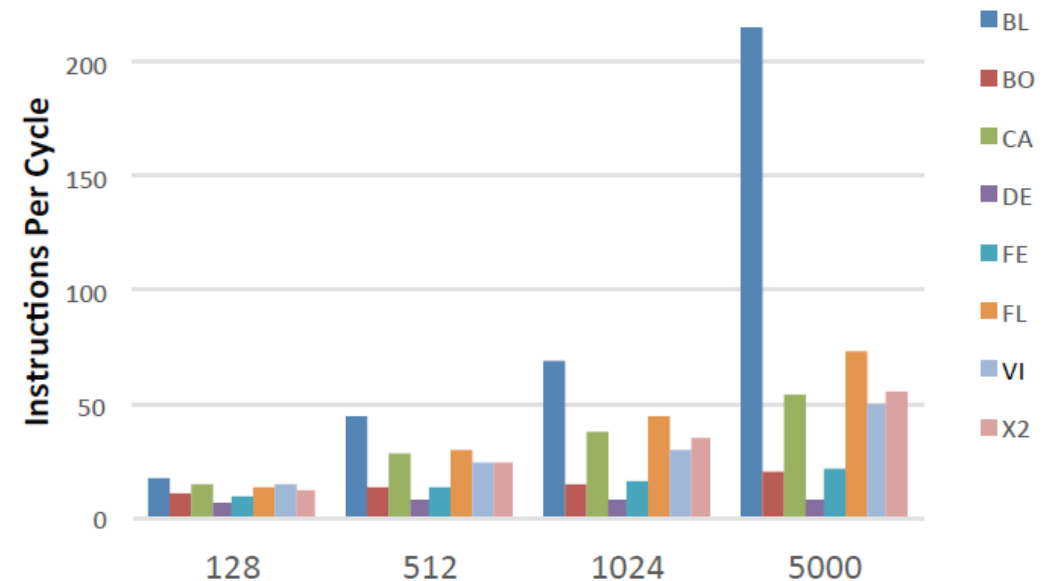
Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

upper bound ILP (2 threads)



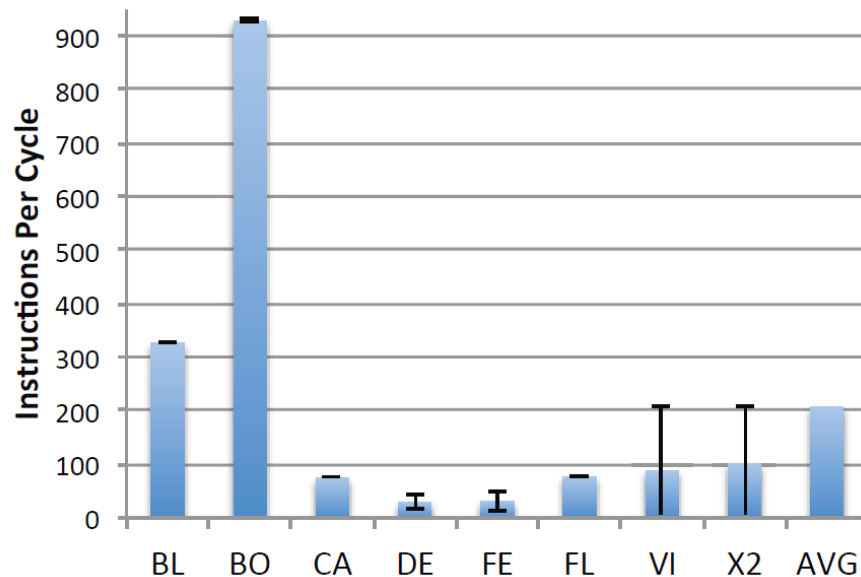
ILP with varying window size (2 threads)



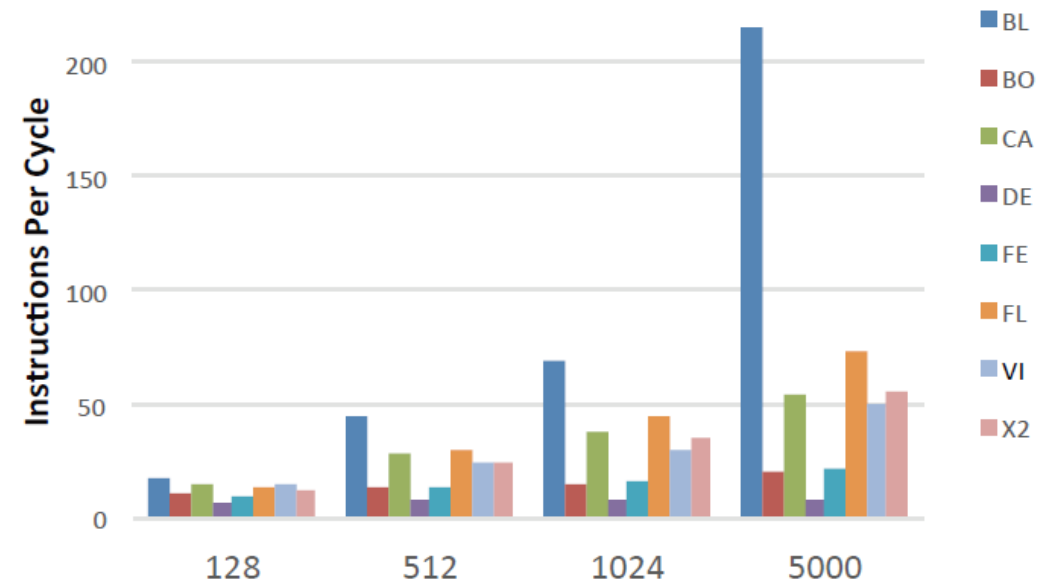
Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

upper bound ILP (2 threads)



ILP with varying window size (2 threads)



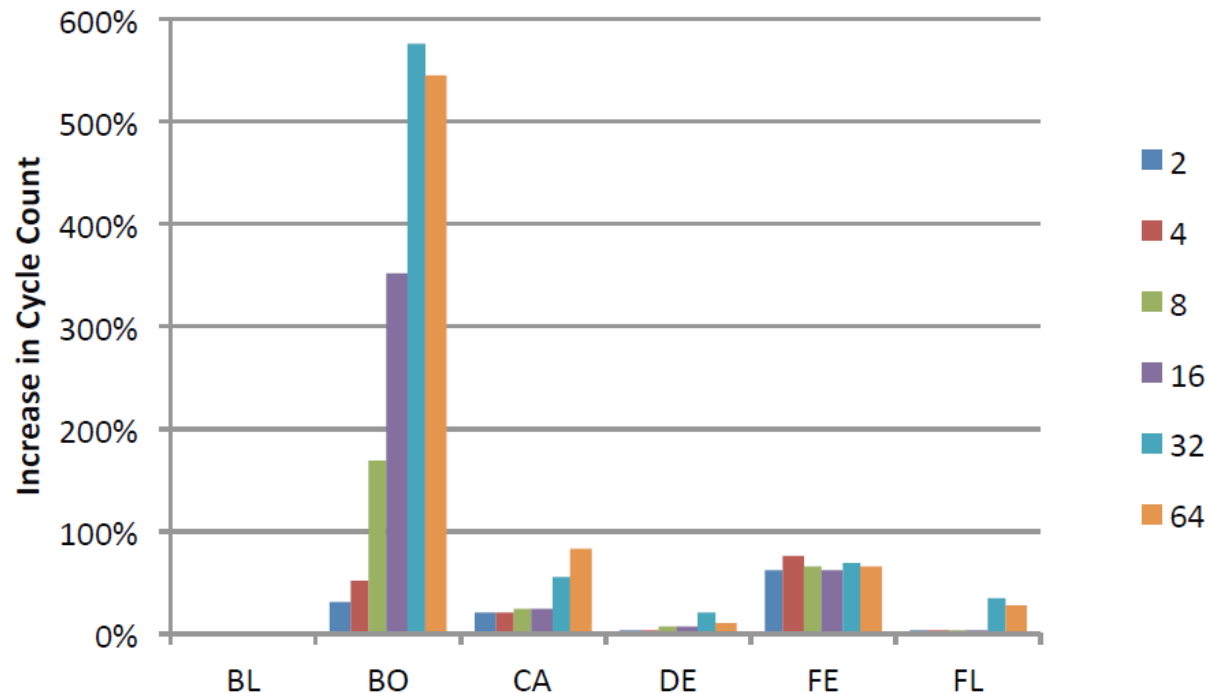
Can think of TLP as merely ILP that's been made explicit by programmer, since instruction windows are not large enough...

Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

increase in critical path length due to synchronization overhead
(varying thread count)

...but synchronization overhead is the cost of making explicit

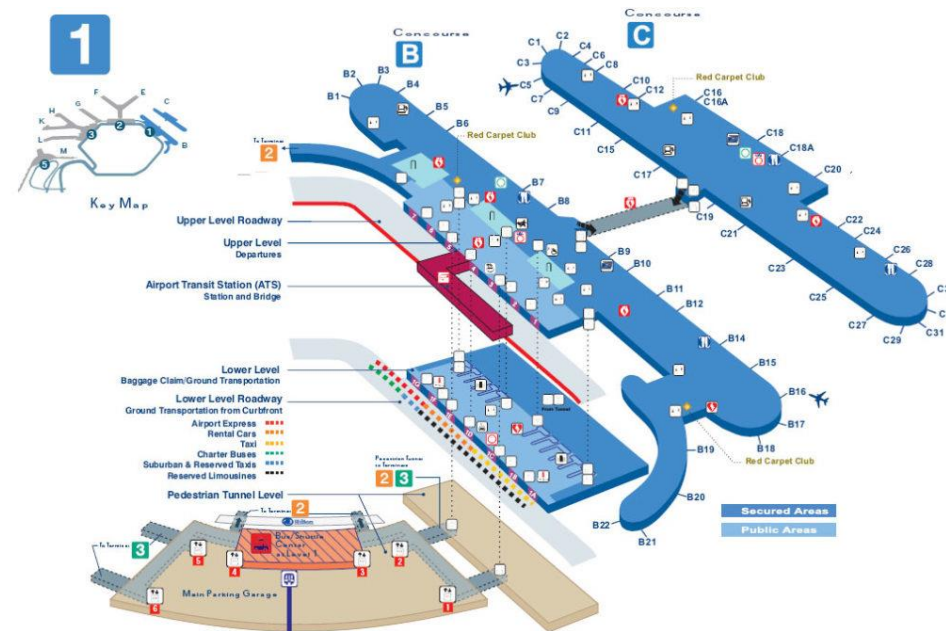


Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

An analogy:

Independent airlines can route through Chicago but limited by size of airport.



Limits on Compute-Level Parallelism

E. Fatehi and P. V. Gratz, *ILP and TLP in Shared Memory Applications: A Limit Study*, PACT, 2014.

An analogy:

Independent airlines can route through Chicago but limited by size of airport.
Add terminals per airline, but high overhead of inter-airline connections.

